Taller de Programación con Estado

Modelos y Paradigmas de Programación Escuela de Ingeniería de Sistemas y Computación - Universidad del Valle

Estudiante: Raúl A. Alzate

Punto 1: Análisis de celdas e igualdad

Código analizado

```
None
declare
A={NewCell 0}
B={NewCell 0}
T1=@A
T2=@B
{Show A==B}   % a)
{Show T1==T2}  % b)
{Show T1=T2}   % c)
A:=@B
{Show A==B}   % d)
```

Análisis paso a paso

Contexto inicial:

- `A = {NewCell 0}`: Crea una celda en memoria con valor inicial 0. `A` es una **referencia** (puntero) a esa celda.
- `B = {NewCell 0}`: Crea **otra** celda diferente en memoria con valor inicial 0. `B` es una referencia distinta a `A`.
- `T1 = @A`: Lee (dereferencía) el contenido de la celda `A`. `T1` se liga al valor `0`.
- `T2 = @B`: Lee el contenido de la celda `B`. `T2` se liga al valor `0`.

Respuestas

a) `{Show A==B}` → `false`

El operador `==` compara **identidad** (igualdad referencial). `A` y `B` son dos celdas distintas creadas con `{NewCell 0}` cada una. Aunque ambas contienen el mismo valor (0), son **objetos diferentes** en el almacén mutable (store). Cada llamada a `{NewCell}` crea una nueva celda con una identidad única. Por lo tanto, `A==B` es `false`.

b) `{Show T1==T2}` → `true`

`T1` y `T2` son variables declarativas (no celdas) ligadas al valor `0`. El operador `==` compara sus valores. Como ambos son `0`, la comparación es `true`. En el modelo declarativo, dos variables ligadas al mismo valor son iguales.

c) `{Show T1=T2}` → `0`

El operador `=` es la **unificación**. `T1=T2` unifica `T1` con `T2`. Dado que ambos ya están ligados al valor `0`, la unificación tiene éxito y la expresión retorna el valor unificado: `0`. El `{Show}` muestra `0`.

d) `{Show A==B}` → `false`

La instrucción `A:=@B` asigna el contenido de la celda `B` (que es `0`) a la celda `A`. Pero `A` ya contenía `0`, así que el contenido no cambia. Sin embargo, lo importante es que `==` compara **identidad de celdas** (referencias), no contenido. `A` y `B` siguen siendo dos celdas diferentes en memoria, por lo que `A==B` sigue siendo `false`. Para que fuera `true`, tendríamos que tener `A` y `B` apuntando a la **misma** celda (por ejemplo, `A = B` antes de crear las celdas, o con aliasing).

Resumen

Marca	**Resultado**	**Razón**
a)	`false`	`A` y `B` son celdas distintas (comparación de identidad)
b)	`true`	`T1` y `T2` son ambos `0` (comparación de valores)
c)	`0`	Unificación exitosa de `0` con `0`, retorna `0`
d)	`false`	`A` y `B` siguen siendo celdas distintas aunque contengan el mismo valor

---

Punto 2: Suma iterativa con celdas (`sum.oz`)

Modelado del problema

Objetivo: Calcular la suma de los enteros desde `A` hasta `B` (ambos incluidos) usando celdas.

Estados:

- `Sum`: celda que acumula la suma parcial.
- `I`: celda que actúa como iterador desde `A` hasta `B`.

Estado inicial: `Sum = 0`, `I = A`

Estado final: `I = B + 1`, `Sum = Σ(k=A..B) k`

Invariante: En cada iteración, `Sum = Σ(k=A..I-1) k`. Es decir, `Sum` contiene la suma de todos los enteros desde `A` hasta `I-1`.

Demostración del invariante:

- **Base (I = A):** `Sum = 0 = Σ(k=A..A-1) k` (suma vacía = 0). ✓
- **Paso inductivo:** Si el invariante se cumple para `I`, entonces después de ejecutar `Sum := @Sum + @I` e `I := @I + 1`, tenemos:
- `Sum_nuevo = Σ(k=A..I-1) k + I = Σ(k=A..I) k`
- `I_nuevo = I + 1`
- El invariante se cumple para `I_nuevo`: `Sum_nuevo = Σ(k=A..I_nuevo-1) k`. ✓

Terminación: `I` incrementa en 1 en cada iteración y la condición de parada es `I > B`, por lo que el ciclo siempre termina (en `B - A + 1` iteraciones).

En el estado final (I = B + 1): Por el invariante, `Sum = Σ(k=A..B) k`, que es el resultado deseado.

Implementación

```
None
declare
fun {Q A B}
  Sum = {NewCell 0}
  I = {NewCell A}
in
  for while:(@I =< B) do
```

```
      Sum := @Sum + @I
      I := @I + 1
   end
   @Sum
end
```

Punto 3: Factorial con celdas (`fact.oz`)

Modelado del problema

Objetivo: Calcular `N!` usando celdas.

Estados:

- `Result`: celda que acumula el producto parcial.
- `I`: celda iteradora desde 1 hasta N.

Estado inicial: `Result = 1`, `I = 1`

Estado final: `I = N + 1`, `Result = N!`

Invariante: `Result = (I-1)!` — en cada iteración, `Result` contiene el factorial de `I-1`.

Demostración del invariante:

- **Base (I = 1):** `Result = 1 = 0!`. ✓
- **Paso inductivo:** Si `Result = (I-1)!`, después de `Result := @Result * @I` e `I := @I + 1`:
- `Result_nuevo = (I-1)! * I = I!`
- `I_nuevo = I + 1`
- `Result_nuevo = (I_nuevo - 1)!`. ✓

Terminación: `I` crece de 1 a `N+1`, siempre termina.

En el estado final (I = N+1): `Result = N!`. ✓

Implementación

```
None
declare
fun {Fact N}
  Result = {NewCell 1}
  I = {NewCell 1}
in
  for while:(@I =< N) do
     Result := @Result * @I
     I := @I + 1
  end
  @Result
end
```

Punto 4: Fibonacci con celdas (`fibo.oz`)

Modelado del problema

Objetivo: Calcular `fib(N)` donde `fib(n) = 1` si `n < 2`, y `fib(n) = fib(n-1) + fib(n-2)` si `n >= 2`.

Estados:

- `Prev`: celda con `fib(I-2)` (el valor anterior anterior).
- `Curr`: celda con `fib(I-1)` (el valor anterior).
- `I`: celda iteradora.

Estado inicial: `Prev = 1` (fib(0)), `Curr = 1` (fib(1)), `I = 2`

Estado final: `I = N + 1`, `Curr = fib(N)`

Invariante: `Prev = fib(I-2)` y `Curr = fib(I-1)`.

Demostración del invariante:

- **Base (I = 2):** `Prev = 1 = fib(0)`, `Curr = 1 = fib(1)`. ✓
- **Paso inductivo:** Después de `Temp = @Curr`, `Curr := @Prev + @Curr`, `Prev := Temp`, `I := @I + 1`:
- `Curr_nuevo = fib(I-2) + fib(I-1) = fib(I)`
- `Prev_nuevo = fib(I-1)`
- `I_nuevo = I + 1`

- `Prev_nuevo = fib(I_nuevo - 2)` y `Curr_nuevo = fib(I_nuevo - 1)`. ✓

Terminación: `I` crece de 2 a `N+1`.

En el estado final (I = N+1): `Curr = fib(N)`. ✓

Implementación

```
None
declare
fun {Fibo N}
  if N < 2 then 1
  else
     Prev = {NewCell 1}
     Curr = {NewCell 1}
     I = {NewCell 2}
  in
     for while:(@I =< N) do
        local Temp = @Curr in
           Curr := @Prev + @Curr
           Prev := Temp
        end
        I := @I + 1
     end
     @Curr
  end
end
```

Punto 5: BubbleSort con celdas (`bubble.oz`)

Modelado del problema

Objetivo: Ordenar una lista de enteros usando el algoritmo BubbleSort con celdas.

Descripción del algoritmo: BubbleSort compara pares de elementos adyacentes y los intercambia si están desordenados. Después de cada pasada completa, el mayor elemento "burbujea" hasta su posición correcta.

Estados:

- `Cells`: tupla de `N` celdas, cada una con un elemento de la lista.
- `I`: iterador externo (pasadas), de 1 a N-1.
- `J`: iterador interno (comparaciones), de 1 a N-I.

Estado inicial: `Cells.K = L.K` para todo `K` en `1..N` (cada celda contiene el elemento original).

Estado final: `Cells` contiene los elementos ordenados de menor a mayor.

Invariante externo (después de la pasada I): Los últimos `I` elementos de `Cells` están en su posición final correcta (son los `I` mayores, ordenados).

Invariante interno (durante la pasada, posición J): `Cells.J` contiene el máximo de `{Cells.1, ..., Cells.J}`.

Terminación: El ciclo externo realiza `N-1` pasadas, el interno disminuye en cada pasada. Total de comparaciones: `N*(N-1)/2`.

Implementación

None

```
declare
fun {BubbleSort L}
  Arr = {List.toTuple arr L}
  N = {Width Arr}
in
  if N =< 1 then L
  else
     local
        Cells = {MakeTuple cells N}
     in
        for K in 1..N do
           Cells.K = {NewCell Arr.K}
        end
        for I in 1..(N-1) do
           for J in 1..(N-I) do
              if @(Cells.J) > @(Cells.(J+1)) then
                 local Temp = @(Cells.J) in
                    (Cells.J) := @(Cells.(J+1))
                    (Cells.(J+1)) := Temp
                 end
              end
           end
        end
        local
           Result = {NewCell nil}
        in
           for K in N..1;~1 do
              Result := @(Cells.K) | @Result
           end
           @Result
```

```
            end
         end
      end
   end
```

Punto 6: Amigos y dinero (`amigos.oz`)

Modelado del problema

Objetivo: Determinar si todas las deudas pueden saldarse pasando dinero únicamente entre personas que aún son amigas.

Observación clave: Las deudas pueden saldarse dentro de un grupo si y solo si la **suma de las deudas dentro de cada componente conexo** del grafo de amistades es igual a 0. Si un componente tiene suma distinta de 0, hay dinero que necesita salir o entrar al componente, pero no hay aristas (amistades) que lo permitan.

Demostración:

- Si la suma de deudas en un componente es 0, entonces hay suficiente dinero entre los miembros del componente para saldar todas las deudas internamente, ya que están todos conectados (directa o transitivamente).
- Si la suma de deudas en un componente es distinta de 0, entonces hay un exceso o déficit que no puede resolverse sin transferencias con personas fuera del componente, lo cual es imposible si no son amigos.

Algoritmo:

1. Encontrar los componentes conexos del grafo de amistades usando BFS con celdas.
2. Para cada componente, sumar las deudas de sus miembros.
3. Si algún componente tiene suma ≠ 0, retornar `false`. Si todos tienen suma 0, retornar `true`.

Estados:

- `Visitado`: tupla de celdas booleanas para BFS.
- `Cola`: celda con la lista de nodos por visitar.
- `Componente`: celda con la lista de nodos del componente actual.
- `Result`: celda booleana con el resultado acumulado.

Estado inicial: Todos los nodos no visitados, `Result = true`.

Estado final: Todos los nodos visitados, `Result = true` si todas las sumas por componente son 0.

Implementación

```
None
declare
fun {SonAmigos Amistades I J}
  case Amistades
  of nil then false
  [] (A#B)|T then
     if (A==I andthen B==J) orelse (A==J andthen B==I) then true
     else {SonAmigos T I J}
     end
  end
end


fun {ComponenteConexo Amistades N Inicio Visitado}
  Cola = {NewCell [Inicio]}
  Componente = {NewCell [Inicio]}
in
  (Visitado.Inicio) := true
  for while:(@Cola \= nil) do
     local Actual = @Cola.1 in
        Cola := @Cola.2
        for K in 1..N do
           if @(Visitado.K) == false andthen {SonAmigos Amistades Actual K}
then
              (Visitado.K) := true
              Cola := {Append @Cola [K]}
              Componente := K | @Componente
           end
        end
     end
  end
  @Componente
end


fun {PagoDeuda Amistades Deudas}
  N = {Width Deudas}
  Visitado = {MakeTuple vis N}
  Result = {NewCell true}
in
  for I in 1..N do
     Visitado.I = {NewCell false}
  end
```

```
   for I in 1..N do
      if @(Visitado.I) == false then
         local
            Comp = {ComponenteConexo Amistades N I Visitado}
            SumaComp = {NewCell 0}
         in
            for C in Comp do
               SumaComp := @SumaComp + Deudas.C
            end
            if @SumaComp \= 0 then
               Result := false
            end
         end
      end
   end
   @Result
end
```

Verificación con los ejemplos

Ejemplo 1: `{PagoDeuda [1#2 2#3 4#5] [100 ~75 ~25 ~42 42]}`

- Componentes conexos: `{1, 2, 3}` y `{4, 5}`
- Suma componente `{1,2,3}`: `100 + (~75) + (~25) = 0` ✓
- Suma componente `{4,5}`: `(~42) + 42 = 0` ✓
- **Resultado:** `true` ✓

Ejemplo 2: `{PagoDeuda [1#3 2#4] [15 20 ~10 ~25]}`

- Componentes conexos: `{1, 3}` y `{2, 4}`
- Suma componente `{1,3}`: `15 + (~10) = 5 ≠ 0` ✗
- **Resultado:** `false` ✓